GENESIS OS / RELATIONSHIP SCORES

Shadow Activation Runtime
Official Companion System Contract v1

Canonical event-label dependency contract merged for the official build set
Detailed engineering specification for runtime activation, dependency gating, payloads, and writeback governance

Purpose. Promote Shadow Activation from a strong standalone ruleset into a governed companion-system contract inside the official Genesis OS build package. This document locks the runtime object model, event-label dependencies, lane discipline, activation thresholds, storage packets, downstream permissions, and quality gates so developers can implement the system without inventing new logic in product code.

Reference alignment: corrected unified master blueprint, workbook-governed scoring stack, cluster atlas, readiness and forecast contracts, Human Emotion Genome registry, Sanctuary contract, and the drift companion contract.

1. Why this document exists

Shadow Activation already exists conceptually across the Genesis OS package: the workbook defines shadow types, the cluster layer explains pattern tension, the forecast layer expects governed instability signals, and Sanctuary provides reflective or healing responses. The missing piece is runtime law. Until this contract, the deterministic activation logic lived too much like a standalone rules engine rather than an official build-set companion system.

This contract closes that gap. It defines exactly what runtime shadow activation is, what evidence is allowed to trigger it, which event labels must exist before activation writes are allowed, how activation decays, how it interacts with drift and clusters, and how the rest of the stack is allowed to consume it.

1.1 What this document is

- A companion-system contract, not a replacement for workbook score law.
- A runtime interpretation and escalation layer that detects when a baseline shadow pattern appears to be behaviorally active in a governed time window.
- A dependency contract requiring canonical event labels before activation writes are allowed.
- A packet and storage specification for downstream services such as forecast, Sanctuary, AI guidance, and calibration.

1.2 What this document is not

- Not a diagnostic or mental-health claim.
- Not permission to overwrite baseline archetype, baseline shadow identity, or workbook scoring tables from a single volatile period.
- Not freeform AI inference. AI may read governed outputs; AI may not mint shadow activations on its own.
- Not an excuse for client-side code to improvise hidden risk logic.

2. Non-negotiable positioning inside the Genesis OS stack

- Shadow Activation is a companion system. It sits beside drift, grief, anger, ExsAnonymous, and the Human Emotion Genome. It is not the compatibility formula itself.
- Baseline shadow_type_id remains workbook-rooted and assessment-rooted. Runtime activation answers a different question: which shadow appears behaviorally active now?
- The system must support both individual_shadow_activation and pair_shadow_pressure views. The first governs user state; the second explains relationship context when both users interact.
- Shadow Activation belongs to the medium lane by default, with tightly limited fast-lane mirrors for UX warnings and no authority to mutate slow-lane identity truth.
- Every activation write must carry provenance, source labels, rules_version, confidence, freshness, and dependency satisfaction status.
- Shadow Activation must decay. It should capture live pressure, not preserve permanent accusations.

3. Operating definition

In Genesis OS, Shadow Activation is the governed runtime family that determines whether a known shadow profile is presently expressing through corroborated behavior, language, pattern repetition, or pair dynamics strongly enough to matter operationally. It is a stateful, time-bounded interpretation built from event labels, cluster evidence, drift signals, messaging patterns, Sanctuary reflections, and governed feature extractions.

3.1 Core runtime question

```
Given the user's baseline shadow profile and recent governed evidence, is a specific
shadow pattern currently active, rising, sustained, resolving, or inactive?
```

3.2 Shadow activation objectives

- Detect meaningful shadow expression early enough to support safer interpretation and intervention.
- Differentiate baseline predisposition from currently expressed pattern.
- Feed forecast and pair-risk logic with auditable, decaying runtime signals.
- Trigger Sanctuary reflections or repair experiences without turning the product into a moral courtroom.
- Create a calibration-ready history of what the engine saw, why it acted, and whether the state later resolved.

4. Canonical shadow families and activation scope

The runtime layer is anchored to the six governed shadow types already defined for Relationship Scores. Activation never invents a seventh shadow in product code; new families must be added only through controlled registry updates.

Shadow family	Runtime expression focus	Typical evidence families	Allowed downstream uses	Never assume from one event
Manipulator	charm used instrumentally; selective truth; relational leverage	messaging tactics, contradiction clusters, repair asymmetry, target-specific pressure	forecast risk, caution guidance, private reflection, admin review	flirty tone alone
Stonewaller	withholding, shutdown, silence as power or avoidance	reply gaps, unresolved repair bids, conflict retreat labels, push-away clusters	drift amplification, forecast drag, Sanctuary repair prompts	one delayed reply
Love Bomber	intensity spikes that outpace stability or consistency	fast escalation, idealization language, pursuit bursts, volatility after intimacy	pace caution, pair forecast, readiness moderation	enthusiastic interest alone
Chameleon	identity or stance shifting to maintain approval or access	value inconsistency, tone mirroring, self-description drift,	integrity risk, readiness friction, reflective prompts	normal adaptation

		contradiction clusters		
Scorekeeper	transactional memory and resentment ledgering	message counts, reciprocity claims, old hurt resurfacing, repair withholding	repair risk, conflict forecasting, guided interventions	one complaint about fairness
Self-Saboteur	connection-damaging moves triggered by closeness, fear, shame, or collapse	retreat after warmth, preemptive exits, cancellation loops, vulnerability contradictions	forecast fragility, Sanctuary grounding, readiness caution	normal caution

5. Event-label dependency model

Runtime shadow activation may only fire from canonical event labels that are registered, typed, freshness-scored, and lane-valid. The engine may consume raw telemetry, but it must not write an activation state until the raw material has been transformed into governed event labels.

5.1 Why event-label dependencies matter

- They give every activation a traceable reason path.
- They prevent one service from using different names than another for the same behavior.
- They let QA and calibration inspect the exact dependency chain that produced activation.
- They stop AI or client code from making up hidden reasons.

5.2 Canonical label classes

Label class	Prefix	Owned by	Examples	Use in activation
Assessment-derived	AS_	workbook + assessment service	AS_SHADOW_LOVEBOMB_TILT, AS_INTEGRITY_LOW, AS_REPAIR_GAP	baseline priors and slow-lane seed
Polarity-derived	PO_	polarity service	PO_PURSUIT_SURGE, PO_TENSION_DEPENDENCE, PO_SURRENDER_UNSAFE	chemistry-context modifiers only
Cluster-derived	CL_	cluster engine	CL_VULNERABILITY_LOOP, CL_PUSH_PULL, CL_CONTROL_AVOIDANCE	strong runtime corroborators
Drift-derived	DR_	drift service	DR_WITHDRAWAL_RISE, DR_REPAIR_SILENCE, DR_ESCALATION_IMBALANCE	time-window trend evidence

Messaging telemetry	MS_	event spine / telemetry normalizer	MS_BID_IGNORED, MS_REPLY_DELAY_SPIKE, MS_BURST_INTIMACY, MS_CANCEL_AFTER_CLOSENESS	behavioral trigger inputs
Sanctuary-derived	SA_	Sanctuary services	SA_EXS_RESIDUE_HIGH, SA_WHISPER_CONTAINMENT_FAIL, SA_MIRROR_ADMISSION_PATTERN	reflective support, limited corroboration
Genome-derived	GE_	HEG extractor registry	GE_CONTRADICTION_DENSITY_HIGH, GE_PERSPECTIVE_SHIFT_FAST, GE_CONTROL_LEXICON_RISE	extractor-grade support with thresholds
Admin / moderation	AD_	trust-review controls	AD_MANUAL_HOLD, AD_FALSE_POSITIVE_SUPPRESS	governance and override only

5.3 Activation dependency rule

```
No shadow activation write may occur unless at least one runtime label from {CL_, DR_, MS_, GE_, SA_} is present and the dependency packet also contains a valid baseline prior from AS_.
```

5.4 Canonical dependency tiers

Tier	Meaning	Minimum requirement	Typical outcome
Tier 0	No activation authority	baseline shadow only; no valid runtime label chain	store seed only
Tier 1	Seed	AS_ + one weak runtime label or one low-confidence GE_ signal	no activation; monitor
Tier 2	Eligible	AS_ + at least two corroborating runtime labels from two families	rising state allowed
Tier 3	Strong	AS_ + cluster or drift corroboration plus repeated behavioral labels	active or sustained allowed
Tier 4	Critical	AS_ + repeated corroboration across multiple windows or safety-significant pattern	sustained/high state, stronger intervention, admin review option

6. Source-to-shadow mapping registry

The registry below defines which source families are allowed to support which runtime shadow patterns. Sources may seed, confirm, soften, or suppress. A label family that is not registered here is not allowed to affect activation in production.

Shadow	Seed sources	Confirming sources	Softening sources	Suppressing sources	Common false positives	Notes
Manipulator	AS_, GE_	CL_, MS_, DR_	SA_ reflection ownership, stable repair labels	AD_, explicit repair consistency	high charisma, strategic communicatio n for work	requires leverage pattern, not mere smoothness
Stonewaller	AS_, DR_	MS_, CL_, GE_	timely repair completion, re-engagement events	AD_, outage/system issues	busy schedule, one long workday	needs conflict-context or repeated withdrawal
Love Bomber	AS_, PO_	MS_, CL_, DR_	pace alignment, stable follow-through	AD_ safety suppress, verified mutual planning	healthy enthusiasm, honeymoon phase	must distinguish intensity from consistency
Chameleon	AS_, GE_	CL_, SA_, MS_	stable values statements over time	AD_ review, onboarding correction	growth or genuine learning	requires identity shift or stance drift evidence
Scorekeeper	AS_	MS_, CL_, GE_, SA_	successful repair closure, gratitude reciprocity labels	AD_ override if context mismatch	reasonable hurt expression	needs repetitive ledger behavior
Self-Saboteur	AS_, CL_	MS_, DR_, SA_, GE_	regulation completion, repaired re-entry	AD_ review if external hardship dominates	healthy caution or boundary setting	best modeled through repeated closeness-then-retreat

7. Runtime state machine

Shadow Activation is stateful. It must move through governed states instead of flipping from off to on. The state machine below is canonical for v1.

State	Meaning	Entry conditions	Exit conditions
inactive	no current activation authority	baseline exists, dependency chain	seed evidence appears

		insufficient	
seeded	possible pattern nominated	Tier 1 dependency packet	decay to inactive or rise to monitoring
monitoring	watch window open, evidence accumulating	repeat low-to-moderate labels within freshness window	insufficient evidence decays or eligible rises
rising	activation becoming credible	Tier 2 met with coherent label family map	sustained if repeated; resolving if softened
active	credible current expression	Tier 3 met; writeback permitted	sustained, resolving, or suppressed
sustained	high-confidence ongoing expression	Tier 3/4 across repeated windows	resolve, decay, or manual hold
resolving	prior activation cooling down	softening and absence of confirming labels for window	inactive after cooldown or re-rise
suppressed	hidden from downstream use except audit	admin hold, dependency failure, false-positive rule, or safety block	manual release or rule correction

7.1 Transition discipline

- inactive -> seeded requires valid AS_ prior plus at least one registered runtime label.
- seeded -> monitoring requires freshness-valid repetition or cross-family corroboration.
- monitoring -> rising requires Tier 2 eligibility and no suppressing label dominance.
- rising -> active requires threshold satisfaction plus at least one strong corroborator from CL_, DR_, or repeated MS_.
- active -> sustained requires persistence across windows or critical dependency stack.
- active/sustained -> resolving requires softening labels and absence of confirming labels for the configured cooldown window.
- Any state -> suppressed may occur when dependency integrity fails, admin hold exists, or false-positive resolution triggers.
- resolving -> inactive requires no reconfirmation and expiration of the configured decay timer.

8. Canonical payload contract

All runtime shadow writes must emit a governed payload. Narrative text alone is insufficient. The contract below should be implemented as a versioned service packet and persisted in an auditable store.

Field	Type	Required	Definition
activation_id	uuid	yes	unique write identifier
user_id	uuid	yes	primary subject of

			activation
pair_id	uuid/null	conditional	included when pair context contributed
shadow_family	enum	yes	one of six governed shadow types
state	enum	yes	inactive/seeded/ monitoring/rising/active/ sustained/resolving/ suppressed
severity_band	enum	yes	none/low/moderate/high/ critical
dependency_tier	int	yes	0 through 4
baseline_prior	object	yes	baseline shadow record and confidence
source_labels	array[label_ref]	yes	canonical event labels used
supporting_clusters	array[string]	no	cluster IDs if relevant
drift_refs	array[string]	no	drift snapshot references if relevant
confidence	decimal	yes	0.00-1.00 governed confidence
freshness_window_hours	int	yes	window used to compute activation
writeback_class	enum	yes	none/light/moderate/ strong/restricted
rules_version	string	yes	contract version applied

8.1 Writeback classes

- none: seed or monitoring state only; may log but cannot affect downstream score families.
- light: allows private guidance and low-impact forecast context.
- moderate: allows forecast friction, Sanctuary prompts, and AI tone moderation.
- strong: allows forecast drag, stronger caution surfaces, and repair-oriented interlocks.
- restricted: stored for audit or admin use but hidden from most product surfaces due to safety or governance limits.

9. Event-label dependencies by shadow family

The following table describes the most important required label patterns for each shadow family. These are dependency hints that should be translated into code tables and threshold references.

Shadow	High-value labels	Required corroboration shape	Softening / exit labels	Downstream caution
Manipulator	MS_BOUNDARY_TEST, GE_LEVERAGE_LANGUAGE, CL_READINESS_OVERSTATEMENT, DR_ASYMMETRIC_REPAIR	needs leverage or strategic inconsistency across at least two families; one charm label never enough	SA_OWNERSHIP_PRESENT, MS_REPAIR_FOLLOWTHROUGH	high false-positive sensitivity
Stonewaller	MS_CONFLICT_SILENCE, DR_REPAIR_SILENCE, CL_REPAIR_GAP, GE_WITHHOLDING_PATTERN	requires withdrawal in conflict or repair context, not generic busyness	MS_REENGAGE_REPAIR, SA_REPAIR_INTENT	protect against work-schedule misreads
Love Bomber	MS_BURST_INTIMACY, PO_PURSUE_SURGE, CL_PUSH_PULL, DR_ESCALATION_IMBALANCE	requires intensity spike plus instability or inconsistency signal	MS_STABLE_FOLLOWTHROUGH, DR_PACE_STABLE	calibrate carefully for new relationships
Chameleon	GE_POSITION_SHIFT_FAST, AS_INTEGRITY_LOW, CL_CONTROL_AVOIDANCE, SA_IDENTITY_MIRROR_NOTE	requires identity or stance drift, not normal learning	GE_VALUES_STABLE, SA_SELF_DEFINITION_CLEAR	avoid punishing healthy flexibility
Scorekeeper	MS_LEDGER_REFERENCE, CL_REPAIR_GAP, GE_REPAYMENT_LANGUAGE, SA_OLD_HURT_LOOP	requires repeated transactional framing across interactions	MS_FORGIVENESS_EVENT, SA_REPAIR_COMPLETE	hurt people may sound sharp; repetition matters
Self-Saboteur	CL_VULNERABILITY_LOOP, MS_CANCEL_AFTER_CLOSENESS, DR_WITHDRAWAL_RISE, SA_FEAR_ADMISSION	requires closeness-then-damage rhythm, not one retreat	MS_SAFE_REENTRY, SA_REGULATION_COMPLETE	most likely to benefit from Sanctuary rather than blunt warnings

10. Threshold and calibration stance

This contract intentionally separates deterministic runtime law from tuneable numeric thresholds. The production system should keep exact score cutoffs, counts, and freshness values in code tables or workbook-adjacent configuration, not hard-coded across multiple services. The following v1 stance is mandatory even before later tuning.

- A single event label must never create active or sustained shadow activation.
- At least one corroborating source family beyond the assessment prior is required for any rising or active state.
- Cluster or drift corroboration dramatically increases activation authority because those services already represent governed interpretations.
- Genome extractor outputs may support activation only when extractor confidence meets published thresholds and the feature is on the canonical registry.
- Sanctuary self-report may support or soften activation but must rarely be the sole confirmer for high-severity states.
- Suppression and false-positive protection must be stronger for Manipulator and Love Bomber because social enthusiasm can be misread.

10.1 Recommended code-table keys

Config key	Purpose	Example value shape
shadow_activation.min_cross_family_sources	minimum corroborating families for rising	integer
shadow_activation.max_seed_age_hours	expiration window for seed evidence	integer
shadow_activation.cluster_bonus_map	how much each cluster family boosts dependency tier	json map
shadow_activation.softening_decay_hours	cooldown rules by shadow family	json map
shadow_activation.false_positive_guardrails	label combinations that suppress activation	json map
shadow_activation.visibility_matrix	what each state may influence downstream	json map
shadow_activation.extractor_threshold_refs	registry pointers to HEG feature thresholds	array of registry IDs

11. Downstream permissions and lane governance

Not every activation state is allowed to do everything. This section prevents runtime shadow logic from quietly rewriting the rest of the product.

State	Readiness	Forecast	Sanctuary	AI tone	User-visible direct label

inactive	none	none	none	none	no
seeded	none	context only	optional reflection seed	none	no
monitoring	light note only	low-confidence caution	soft prompt allowed	gentle tone softening	no
rising	light friction	moderate context	targeted prompt allowed	moderate caution	usually no
active	moderate friction	strong context	guided intervention allowed	guarded interpretation	private softened language only
sustained	moderate, never identity overwrite	strong or critical context	strong repair/healing routes	heightened caution + containment	private softened language only
resolving	reduced friction	fading context	closure or grounding prompts	warmer tone	optional non-shadow phrasing
suppressed	none public	audit only	manual-only if needed	AI blocked or limited	no

11.1 Core lane rules

- Slow lane: baseline shadow identity, archetype, and canonical workbook traits.
- Medium lane: active shadow states, forecast context, pair caution, repair guidance, cluster interplay.
- Fast lane: ephemeral UI handling such as warning banners, temporary tone moderation, or one-session prompts. Fast lane values must never masquerade as identity truth.
- No lane may bypass rules_version, dependency proofs, or audit logs.

12. Interlock contracts

12.1 Cluster interlock

- Cluster engine may nominate, confirm, or soften shadow activation through registered CL_ labels.
- Clusters such as Vulnerability Loop, Push-Pull, Repair Gap, Control Avoidance, Readiness Overstatement, and Slow Drift are especially relevant.
- Shadow activation may read clusters; cluster engine may not directly set active shadow state without the activation contract.

12.2 Drift interlock

- Drift snapshots may elevate dependency tier when they show persistent withdrawal, repair silence, escalation imbalance, or closeness-then-distance patterns.
- Drift never overwrites shadow family; it supplies trend evidence.
- The drift contract remains authoritative for rollup metrics and freshness windows.

12.3 Sanctuary interlock

- Sanctuary may receive activation context to shape prompts, but it should translate shadow states into humane reflective language.
- ExsAnonymous, Emotional Mirror, Whisper Window, and Archive of Self may all create SA_ labels that support softening, insight, or governed corroboration.
- Sanctuary sessions should never auto-upgrade a user to sustained activation without corroborating non-Sanctuary evidence.

12.4 Human Emotion Genome interlock

- Only registered extractors with published thresholds may generate GE_ labels.
- Genome features enrich activation confidence and contradiction density analysis.
- Genome outputs are support signals under governance, not freeform identity writers.

12.5 AI interlock

- AI may consume final payloads, not raw internal heuristics unless specifically authorized.
- AI may explain patterns in softened language; it may not expose hidden shadow labels to other users.
- AI may not independently mint, escalate, or suppress activation states.

13. Storage objects and API surface

Developers should implement Shadow Activation as a versioned service with explicit read and write objects rather than sprinkling booleans across user records.

13.1 Recommended objects

- shadow_activation_event: append-only raw write object with dependency packet and audit fields.
- shadow_activation_snapshot: latest subject-level summarized state per shadow family.
- pair_shadow_pressure_snapshot: pair-context view derived from one or both partners.
- shadow_activation_resolution_event: records softening, suppression, manual clear, or decay completion.

13.2 Suggested endpoints

- POST /shadow-activation/evaluate - evaluates a subject against registered dependency packet(s).
- GET /shadow-activation/snapshot/{user_id} - current subject summary.
- GET /shadow-activation/pair/{pair_id} - pair shadow pressure summary if allowed.
- POST /shadow-activation/admin/suppress - manual governance hold.
- POST /shadow-activation/admin/release - release after review.
- GET /shadow-activation/audit/{activation_id} - full provenance trail.

13.3 Minimum audit fields

Field	Why it exists	Field	Why it exists
rules_version	reproducibility	dependency_hash	proof of source set
source_count	traceability	cross_family_count	eligibility check

freshness_score	staleness control	suppression_reason	governance explainability
computed_at	ordering	computed_by_service	ownership
subject_scope	individual or pair	visibility_scope	surface control

14. QA scenarios and build sequence

Shadow Activation should be rolled out like a controlled power system, not a surprise patch. The QA matrix must include both true-positive and false-positive tests for every shadow family and every major dependency chain.

14.1 Required QA scenarios

- Single intense but healthy enthusiasm should not activate Love Bomber.
- One long reply delay without conflict context should not activate Stonewaller.
- A clear self-reflective Sanctuary admission without corroboration should not activate Manipulator at high severity.
- Repeated closeness-then-retreat plus vulnerability cluster plus drift rise should elevate Self-Saboteur.
- Repair completion and stable follow-through should move active or rising states toward resolving.
- Manual suppression should remove downstream visibility without deleting audit truth.

14.2 Implementation order

- Publish canonical event-label registry.
- Lock source-to-shadow dependency table in configuration.
- Implement append-only activation_event store.
- Implement evaluator service and state machine.
- Integrate forecast and Sanctuary consumers behind feature flags.
- Add AI consumption only after audit trails and user-facing copy are approved.
- Run shadow-family QA packs before any public-facing rollout.

14.3 Final contract stance

This companion contract is implementation-ready for the core build set. Future appendices should maintain the exact code-table thresholds, user-facing copy library, shadow-family QA packs, and migration notes when rules_version changes. Workbook truth remains the foundation, while Shadow Activation becomes a governed runtime lens for present-tense behavior rather than an uncontrolled storyteller.